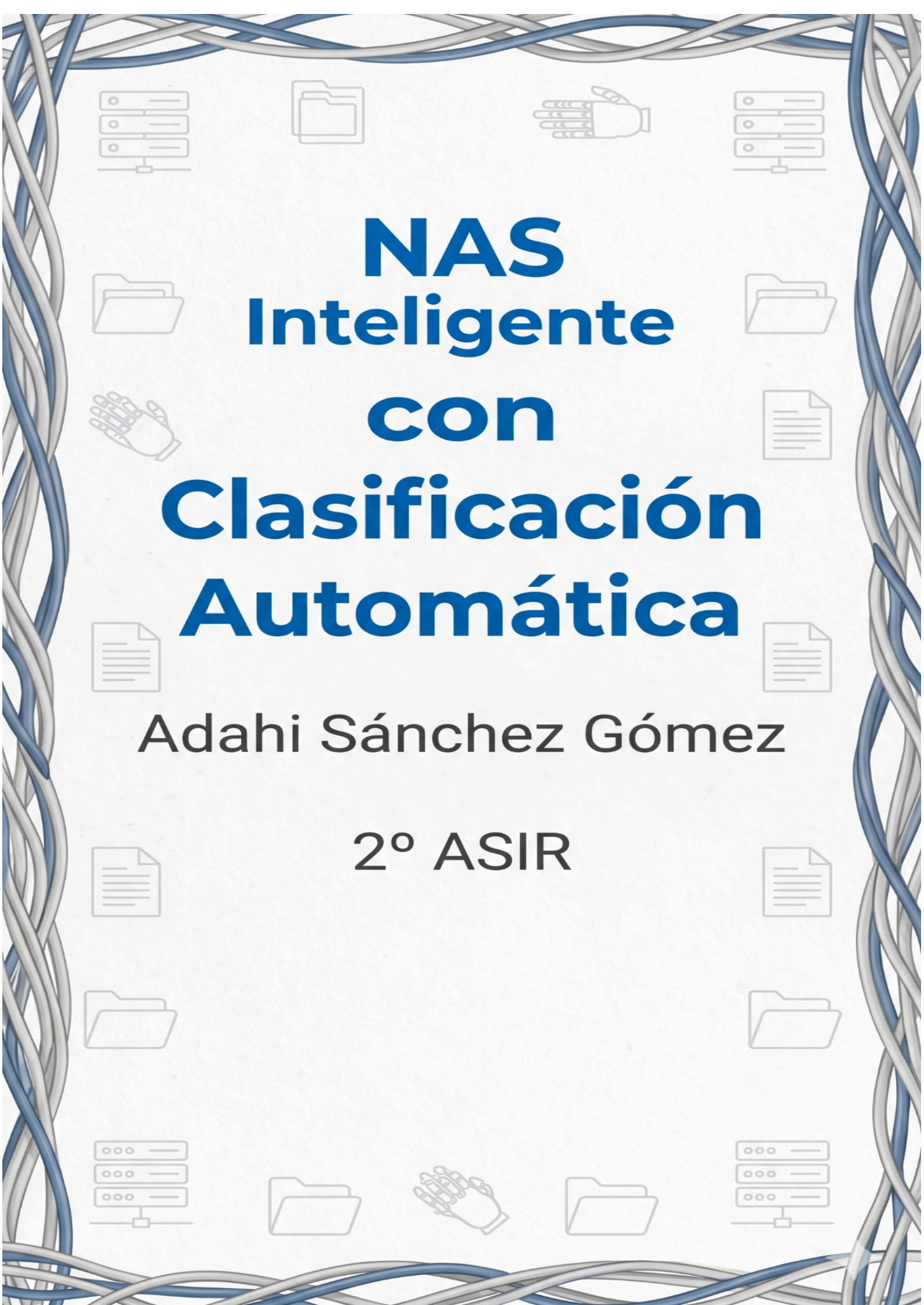




NAS Inteligente con Clasificación Automática

Adahi Sánchez Gómez

2º ASIR



Índice

Índice.....	2
Introducción.....	3
Análisis de Capturas y Proceso de Implementación.....	3
Fase 1: Configuración de Permisos y Estructura de Datos.....	3
Acceso y Verificación del Sistema.....	3
Identificación de Red.....	3
Mantenimiento del Servidor:.....	4
Instalación de la Infraestructura de Contenedores:.....	4
Configuración de Permisos y Estructura.....	5
Definición del Servicio de Red Samba.....	5
Fase 2: Despliegue del Servidor de Red (Samba) y Debugging.....	5
Edición del archivo docker-compose.yml.....	5
Descarga y arranque de servicios.....	6
Troubleshooting de variables y permisos.....	6
Verificación del estado del contenedor.....	6
Intento de conexión y error de credenciales.....	7
Acceso exitoso al recurso compartido nas-ia.....	7
Fase 3: Desarrollo y Dockerización del Cerebro de IA.....	8
Creación del script de Inteligencia Artificial.....	8
Programación de la lógica del organizador.....	8
Creación del entorno de la IA con Dockerfile.....	9
Configuración del Dockerfile para el motor de IA.....	9
Fase 4: Orquestación del Sistema Completo y Troubleshooting.....	10
Orquestación final con Docker Compose.....	10
Construcción de la imagen de la IA.....	10
Resolución de errores de arranque y estado del Firewall.....	11
Ajuste de permisos y depuración del código Python.....	11
Reinicio del entorno para aplicar cambios.....	12
Configuración de contraseñas Samba y reconstrucción.....	12
Confirmación del estado healthy de Samba.....	12
Actualización de herramientas de orquestación.....	13
Resolución de conflictos y limpieza de contenedores.....	13
Despliegue definitivo y exitoso de los servicios.....	14
Fase 5: Pruebas de Clasificación Automática y Validación Real.....	14
Prueba de carga en la carpeta 'inbox'.....	14
Éxito en la clasificación automática.....	15
Optimización de logs en tiempo real.....	15
Segunda prueba de clasificación con contratos.....	16
Verificación de la clasificación de contratos.....	16
Monitorización de logs y confirmación de la IA.....	17

Introducción

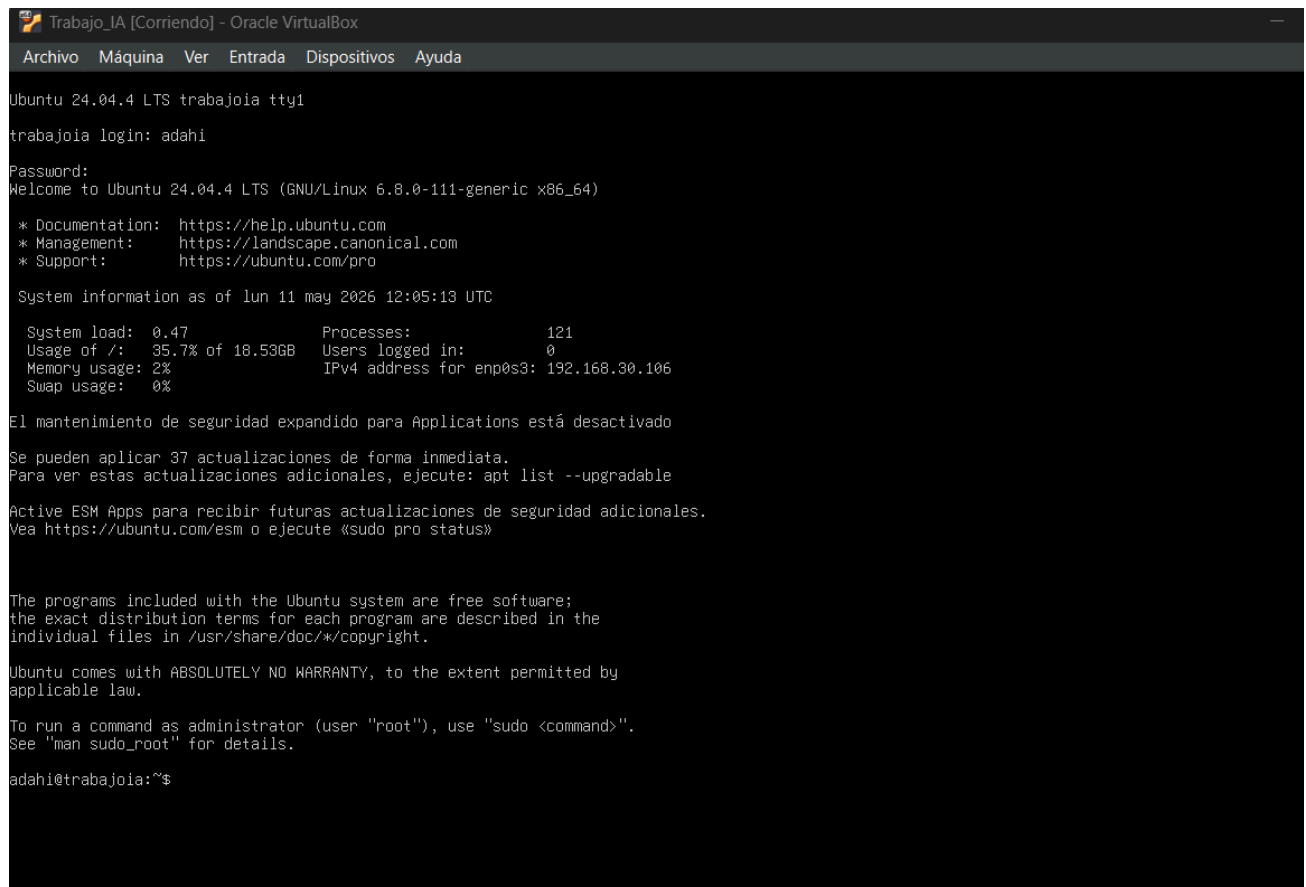
Este proyecto consiste en montar un servidor de almacenamiento en red (NAS) que no sea solo un sitio donde guardar archivos. Lo que he hecho es integrarle un motor de Inteligencia Artificial (OCR) que vigila la carpeta de entrada en tiempo real. El objetivo es que, cuando subamos una imagen, el sistema la lea, sepa si es una factura o un contrato y la mueva automáticamente a su sitio.

Análisis de Capturas y Proceso de Implementación

Fase 1: Configuración de Permisos y Estructura de Datos

Acceso y Verificación del Sistema.

Arranco la máquina virtual en VirtualBox e inicio sesión con mi usuario adahi. En este paso confirmo que el sistema Ubuntu 24.04 está operativo y que las credenciales funcionan bien para empezar con el despliegue.



```
Trabajo_IA [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

Ubuntu 24.04.4 LTS trabajoia tty1
trabajoia login: adahi
Password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of lun 11 may 2026 12:05:13 UTC

System load:  0.47          Processes:            121
Usage of /:   35.7% of 18.53GB Users logged in:          0
Memory usage: 2%           IPv4 address for enp0s3: 192.168.30.106
Swap usage:   0%

El mantenimiento de seguridad expandido para Applications está desactivado
Se pueden aplicar 37 actualizaciones de forma inmediata.
Para ver estas actualizaciones adicionales, ejecute: apt list --upgradable

Active ESM Apps para recibir futuras actualizaciones de seguridad adicionales.
Vea https://ubuntu.com/esm o ejecute «sudo pro status»

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

adahi@trabajoia:~$
```

Identificación de Red

Lanzó el comando `ip a` para localizar la dirección IPv4 del servidor, que es la 192.168.30.106. Este dato es fundamental porque es la dirección que tendré que poner en Windows para conectar con el NAS más adelante.


```
adahi@trabajoia:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:60:54:43 brd ff:ff:ff:ff:ff:ff
    inet 192.168.30.106/24 metric 100 brd 192.168.30.255 scope global dynamic enp0s3
        valid_lft 604669sec preferred_lft 604669sec
    inet6 fe80::a00:27ff:fe60:5443/64 scope link
        valid_lft forever preferred_lft forever
adahi@trabajoia:~$
```

Mantenimiento del Servidor:

Ejecuto el comando `sudo apt update && sudo apt upgrade -y` para poner el sistema al día. Es un paso necesario para que todo el software y las librerías de Docker que instalaré después no den problemas de compatibilidad.

```
adahi@trabajoia:~$
adahi@trabajoia:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for adahi:
Obj:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Obj:2 http://archive.ubuntu.com/ubuntu noble InRelease
Obj:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Des:5 http://archive.ubuntu.com/ubuntu noble/main Translation-es [325 kB]
Des:6 http://archive.ubuntu.com/ubuntu noble/restricted Translation-es [816 B]
Des:7 http://archive.ubuntu.com/ubuntu noble/universe Translation-es [1.371 kB]
Error: NoClassDefFoundError
```

Instalación de la Infraestructura de Contenedores:

Instalo Docker y Docker Compose con `sudo apt install` para tener lista la base de los contenedores. Estas herramientas son las que me permiten separar el servicio del NAS de la IA, trabajando con microservicios para que el despliegue sea más sencillo .

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
adahi@trabajoia:~$ sudo apt install docker.io docker-compose -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  bridge-utils containerd dns-root-data dnsmasq-base pigz python3-compose python3-docker python3-dockerpty
  python3-docker python3-dockerpty python3-docker python3-dotenv python3-texttable python3-websocket runc ubuntu-fan
Paquetes sugeridos:
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2 docker-doc rinse
  zfs-fuse | zfsutils
Se instalarán los siguientes paquetes NUEVOS:
  bridge-utils containerd dns-root-data dnsmasq-base docker-compose docker.io pigz python3-compose python3-docker
  python3-dockerpty python3-docker python3-dotenv python3-texttable python3-websocket runc ubuntu-fan
0 actualizados, 16 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 74,1 MB de archivos.
Se utilizarán 280 MB de espacio de disco adicional después de esta operación.
```

Configuración de Permisos y Estructura

Añado mi usuario al grupo 'docker' con el comando sudo usermod para poder manejar los contenedores más cómodamente sin tener que escribir sudo todo el rato. Después uso newgrp docker para que el cambio se aplique al momento en la sesión actual de la terminal.

```
4.: command not found
adahi@trabajoia:~$ sudo usermod -aG docker $USER
e newgrp docker
adahi@trabajoia:~$
```

Definición del Servicio de Red Samba

Uso el comando mkdir -p para crear de golpe todas las carpetas de datos que usaremos: inbox, facturas, contratos y otros. Después, abro el editor nano para empezar a configurar el archivo docker-compose.yml, donde definiré los puertos y los volúmenes de nuestro servidor Samba .

```
adahi@trabajoia: ~
adahi@trabajoia:~$ mkdir -p ~/nas_ia/datos/{inbox,facturas,contratos,otros}
adahi@trabajoia:~$ nano ~/nas_ia/docker-compose.yml
```

Fase 2: Despliegue del Servidor de Red (Samba) y Debugging

Edición del archivo docker-compose.yml

Preparo el archivo de configuración indicando que use la imagen de Samba y mapee el puerto 445 para la conexión de red. Además, incluyo el comando necesario para crear mi usuario "adahi" y el recurso compartido "NAS-IA" con acceso total.

```
adahi@trabajoia: ~/nas_ia
GNU nano 7.2 /home/adahi/nas_ia/docker-compose.
version: '3.8'

services:
  samba:
    image: dperson/samba
    container_name: nas_samba
    ports:
      - "139:139"
      - "445:445"
    volumes:
      - ./datos:/mnt/nas
    command: -u "adahi;contrasena" -s "NAS-IA;/mnt/nas;yes;no;no;adahi"
    restart: always
```

Descarga y arranque de servicios

Lanzo el entorno con el comando docker-compose up -d, lo que inicia la descarga de las capas de la imagen y crea la red interna. El proceso termina correctamente cuando el contenedor de Samba aparece marcado como "done".

```
adahi@trabajoia: ~/nas_ia
adahi@trabajoia:~$ cd nas_ia/
adahi@trabajoia:~/nas_ia$ docker-compose up -d
Creating network "nas_ia_default" with the default driver
Pulling samba (dperson/samba:latest)...
latest: Pulling from dperson/samba
02fdab01eee5: Pull complete
df20fa9351a1: Pull complete
cdd17995a233: Pull complete
Digest: sha256:66088b78a19810dd1457a8f39340e95e663c728083efa5fe7dc0d40b2478e869
Status: Downloaded newer image for dperson/samba:latest
Creating nas_samba ... done
adahi@trabajoia:~/nas_ia$
```

Troubleshooting de variables y permisos

Al ver que el NAS se reiniciaba, revisé los logs y encontré un fallo de variables en el script de arranque de la imagen. Lo arreglé dándole la propiedad de todas las carpetas de datos a mi usuario mediante el comando chown.

```
adahi@trabajoia:~/nas_ia$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS      NAMES
d088ba897278   dperson/samba  "/sbin/tini -- /usr/_..." About a minute ago   Restarting (1) 1 second ago           nas_samba

adahi@trabajoia:~/nas_ia$ docker logs nas_samba
/usr/bin/samba.sh: line 160: $?: unbound variable
/usr/bin/samba.sh: line 160: $?: unbound variable
/usr/bin/samba.sh: line 160: $?: unbound variable
/usr/bin/samba.sh: line 160: $?: unbound variable
/usr/bin/samba.sh: line 160: $?: unbound variable
/usr/bin/samba.sh: line 160: $?: unbound variable
/usr/bin/samba.sh: line 160: $?: unbound variable
/usr/bin/samba.sh: line 160: $?: unbound variable
/usr/bin/samba.sh: line 160: $?: unbound variable
/usr/bin/samba.sh: line 160: $?: unbound variable
/usr/bin/samba.sh: line 160: $?: unbound variable
adahi@trabajoia:~/nas_ia$ nano ~/nas_ia/docker-compose.yml
adahi@trabajoia:~/nas_ia$ nano ~/nas_ia/docker-compose.yml
adahi@trabajoia:~/nas_ia$ # Asegurarnos de que las carpetas están creadas
mkdir -p ~/nas_ia/datos/{inbox,facturas,contratos,otros}

# Darle la propiedad a tu usuario
sudo chown -R $USER:$USER ~/nas_ia/datos
adahi@trabajoia:~/nas_ia$
```

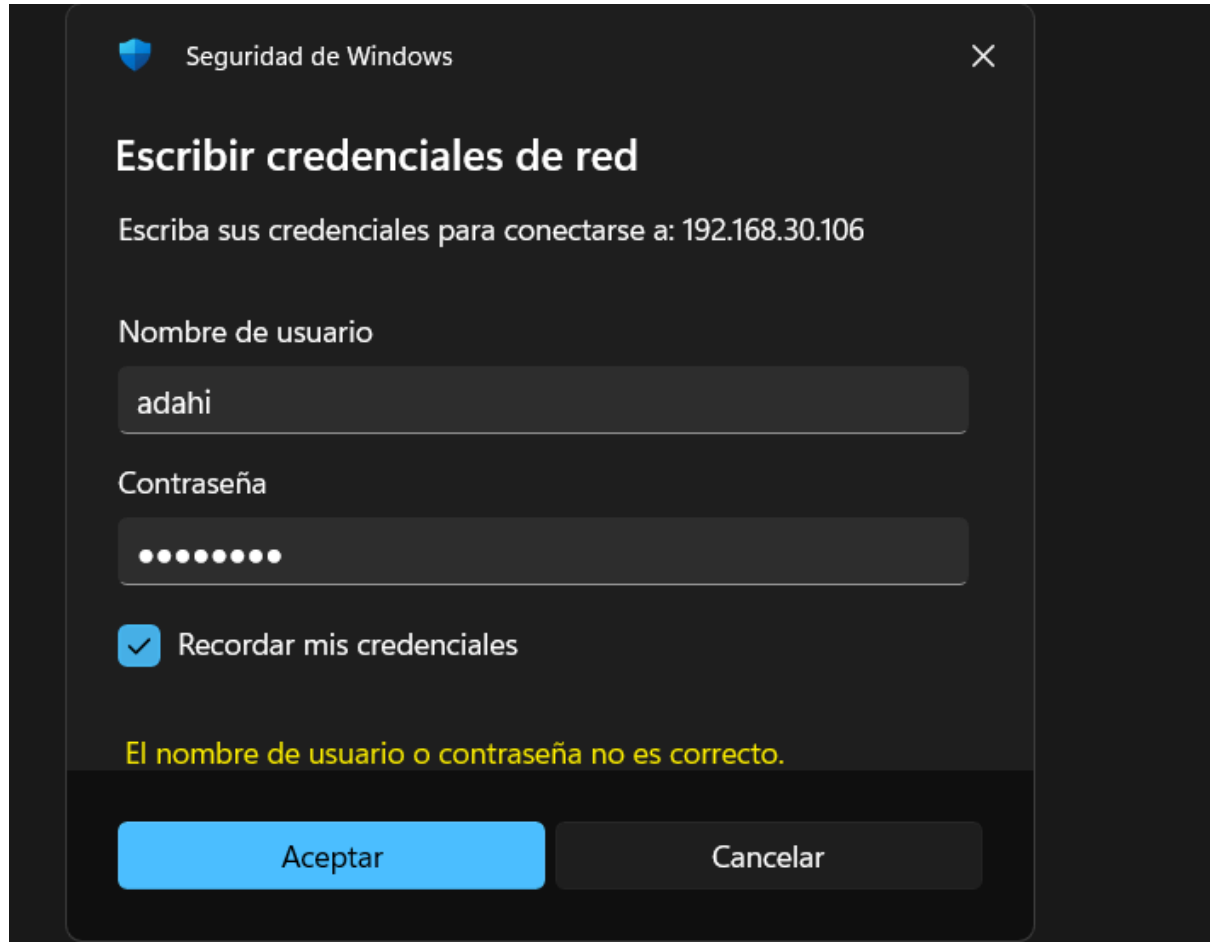
Verificación del estado del contenedor

Lanzo un docker ps para asegurarme de que el contenedor de Samba está activo y funcionando correctamente. Puedo comprobar que los puertos 139 y 445 están abiertos y que el estado marca que el servicio ya está arrancando sin fallos.

```
adahi@trabajoia:~/nas_ia$ docker ps
CONTAINER ID   IMAGE      COMMAND                  NAMES
e888a27633cc   dperson/samba  "/sbin/tini -- /usr/_..." 23 seconds ago   Up 22 seconds (health: starting) 0.0.0.0:139->139/tcp, [::]:139->139/tcp, 137-138/udp, 0.0.0.0:445->445/tcp, [::]:445->445/tcp   nas_samba
adahi@trabajoia:~/nas_ia$
```

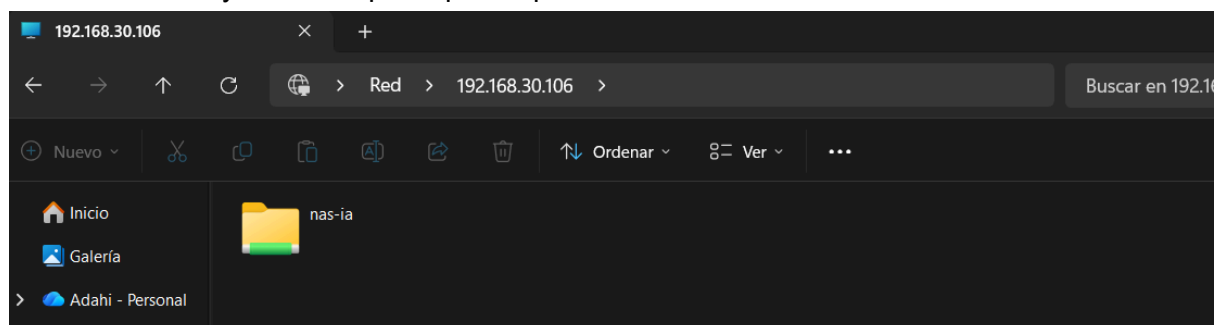
Intento de conexión y error de credenciales

Trato de acceder al recurso compartido desde mi PC usando la IP del servidor, pero me da un error de usuario o contraseña. Esto es parte del proceso de depuración, ya que toca revisar si las credenciales de Samba coinciden exactamente con las que he configurado en el contenedor.



Acceso exitoso al recurso compartido nas-ia

Tras corregir los problemas de login, ya puedo entrar desde el explorador de archivos de Windows usando la IP del servidor. En esta imagen se confirma que el recurso compartido nas-ia es visible y está listo para que empecemos a volcar los archivos.



Fase 3: Desarrollo y Dockerización del Cerebro de IA

Creación del script de Inteligencia Artificial

Entró en el directorio del proyecto y abro el editor nano para empezar a escribir el código de organizador.py . Este archivo es el "cerebro" del sistema, encargado de vigilar la carpeta de entrada y ejecutar la lógica de reconocimiento para mover los documentos.

```
adahi@trabajoia: ~/nas_ia
adahi@trabajoia:~/nas_ia$ cd ~/nas_ia
nano organizador.py
```

Programación de la lógica del organizador

Escribo el código en Python que se encarga de todo el trabajo sucio: usa watchdog para no quitarle el ojo a la carpeta de entrada y pytesseract para leer las imágenes . He programado una lógica sencilla para que, si encuentra palabras como "factura" o "contrato", mueva el archivo automáticamente a su carpeta correspondiente usando la librería shutil .

```
organizador.py
import os
import time
import shutil
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import pytesseract
from PIL import Image

# Configuración de rutas
PATH_INBOX = "/app/datos/inbox"
PATH_BASE = "/app/datos"

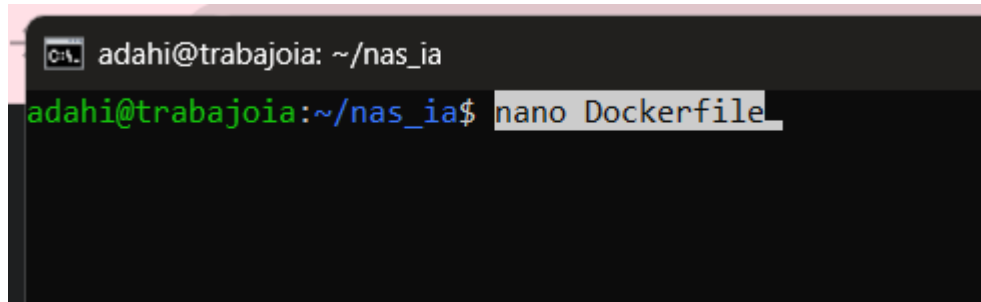
class ProcesadorArchivo(FileSystemEventHandler):
    def on_created(self, event):
        if not event.is_directory:
            print(f"Detectado nuevo archivo: {event.src_path}")
            # Esperar un poco para asegurar que el archivo se terminó de copiar
            time.sleep(2)
            self.clasificar(event.src_path)

def clasificar(self, ruta_archivo):
    try:
        # 1. Leer el texto de la imagen (IA OCR)
        texto = pytesseract.image_to_string(Image.open(ruta_archivo)).lower()
        nombre_archivo = os.path.basename(ruta_archivo)
        # 2. Lógica de clasificación (Palabras clave)
        destino = "otros"
        if any(palabra in texto for palabra in ["factura", "total", "iban", "precio", "cif"]):
            destino = "facturas"
        elif any(palabra in texto for palabra in ["contrato", "acuerdo", "firmado", "clausula"]):
            destino = "contratos"
        # 3. Mover el archivo
        ruta_destino = os.path.join(PATH_BASE, destino, nombre_archivo)
        shutil.move(ruta_archivo, ruta_destino)
        print(f"Archivo {nombre_archivo} movido a -> {destino}")
    except Exception as e:
        print(f"Error procesando {ruta_archivo}: {e}")

if __name__ == "__main__":
    observer = Observer()
    observer.schedule(ProcesadorArchivo(), PATH_INBOX, recursive=False)
    print("IA Vigilando la carpeta inbox...")
    observer.start()
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        observer.stop()
    observer.join()
```


Creación del entorno de la IA con Dockerfile

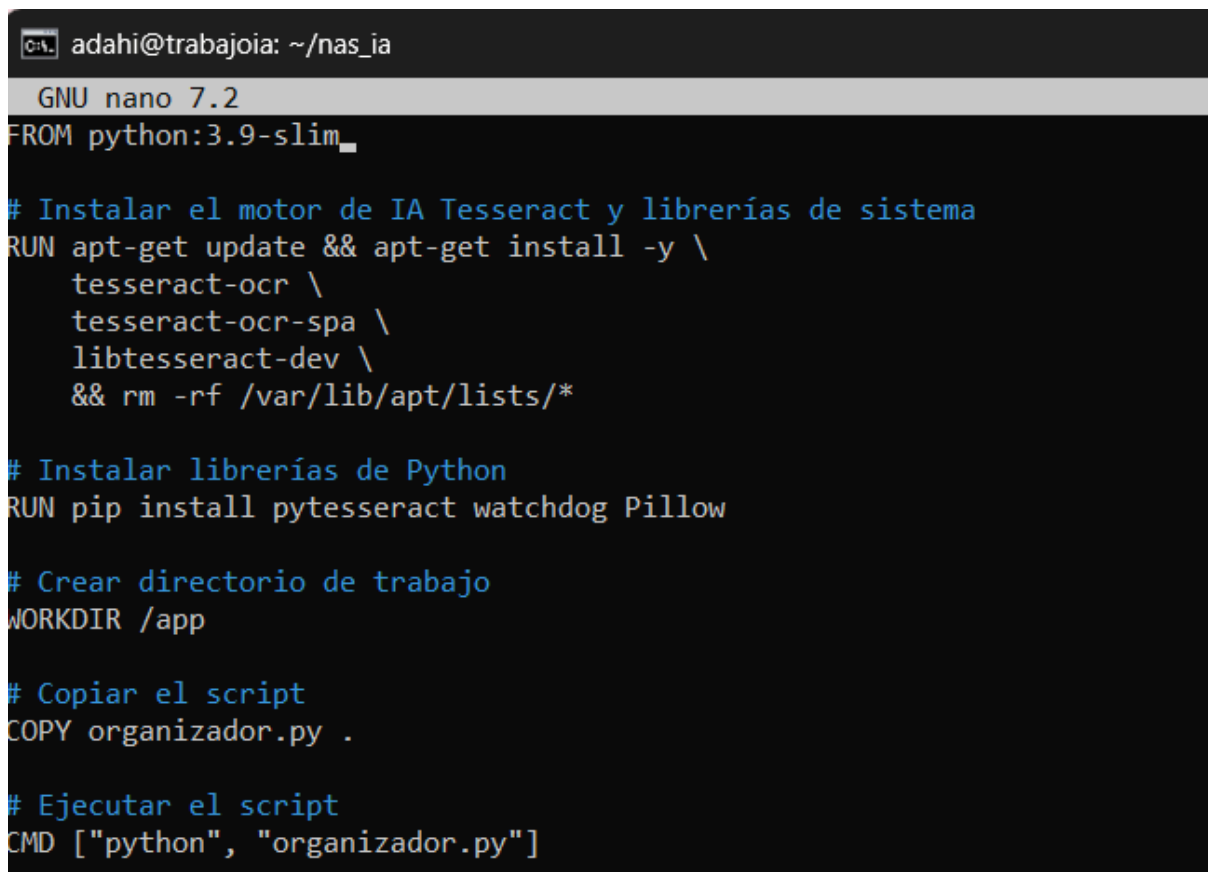
Abro el archivo Dockerfile con el editor nano para empezar a configurar el contenedor de la IA. En este paso es donde defino qué imagen de Python vamos a usar y qué herramientas extra necesita el sistema para poder leer el texto de las imágenes .



```
adahi@trabajoia: ~/nas_ia
adahi@trabajoia:~/nas_ia$ nano Dockerfile
```

Configuración del Dockerfile para el motor de IA

Configuro el Dockerfile instalando el motor Tesseract OCR en español y las librerías necesarias como pytesseract y watchdog . También establezco el directorio de trabajo, copio el script organizador.py y defino el comando para que la IA se ejecute automáticamente al arrancar el contenedor .



```
adahi@trabajoia: ~/nas_ia
GNU nano 7.2
FROM python:3.9-slim

# Instalar el motor de IA Tesseract y librerías de sistema
RUN apt-get update && apt-get install -y \
    tesseract-ocr \
    tesseract-ocr-spa \
    libtesseract-dev \
    && rm -rf /var/lib/apt/lists/*

# Instalar librerías de Python
RUN pip install pytesseract watchdog Pillow

# Crear directorio de trabajo
WORKDIR /app

# Copiar el script
COPY organizador.py .

# Ejecutar el script
CMD ["python", "organizador.py"]
```

Fase 4: Orquestación del Sistema Completo y Troubleshooting

Orquestación final con Docker Compose

Uso el comando cat para escribir el archivo docker-compose.yml definitivo, uniendo de una vez los dos servicios: el NAS (Samba) y el cerebro de la IA . Configuro los volúmenes compartidos para que ambos contenedores vean la misma carpeta de datos, permitiendo que la IA procese en tiempo real lo que llega por red .

```
adahi@trabajoia:~/nas_ia$ cat << 'EOF' > ~/nas_ia/docker-compose.yml
services:
  samba:
    image: dperson/samba
    container_name: nas_samba
    ports:
      - "139:139"
      - "445:445"
    volumes:
      - ./datos:/mnt/nas
    environment:
      USER: "adahi:adahi"
      SHARE: "NAS-IA;/mnt/nas;yes;no;no;adahi"
      USERID: 1000
      GROUPID: 1000
    restart: always
  ia:
    build: .
    container_name: nas_ia_brain
    volumes:
      - ./datos:/app/datos
    restart: always
EOF
adahi@trabajoia:~/nas_ia$
```

Construcción de la imagen de la IA

Lanzo el comando docker-compose up -d --build para que Docker empiece a montar la imagen de la IA paso a paso. Aquí se ve cómo se descarga la base de Python y se preparan todas las capas que configuramos antes para que el sistema pueda arrancar.

```
adahi@trabajoia:~/nas_ia$ docker-compose up -d --build
Building ia
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  8.192kB
Step 1/6 : FROM python:3.9-slim
3.9-slim: Pulling from library/python
b3ec39b36ae8: Pulling fs layer
ea56f685404a: Pulling fs layer
fc7443084902: Pulling fs layer
38513bd72563: Pulling fs layer
a5e5b1b19090: Download complete
ea56f685404a: Download complete
967a6079ce08: Download complete
b3ec39b36ae8: Download complete
38513bd72563: Download complete
fc7443084902: Download complete
38513bd72563: Pull complete
b3ec39b36ae8: Pull complete
```

Resolución de errores de arranque y estado del Firewall

Veo que el contenedor de la IA no arranca y se queda reiniciando todo el rato, así que me toca revisar qué está pasando con ese error de `KeyError` . Aprovecho para reiniciar el Samba y miro con `sudo ufw status` que el firewall esté quitado para que no me dé problemas con la red del servidor .

```

KeyError: 'ContainerConfig'
adahi@trabajoia:~/nas_ia$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS      PORTS      NAMES
bb0e3ea56570   nas_ia_ia  "python organizador..." 2 minutes ago  Restarting (1) 27 seconds ago      nas_ia_brain
adahi@trabajoia:~/nas_ia$ docker-compose restart samba
Restarting e888a27633cc_nas_samba ... done
adahi@trabajoia:~/nas_ia$ sudo ufw status
Status: inactive
adahi@trabajoia:~/nas_ia$

```

Ajuste de permisos y depuración del código Python

Uso el comando `sudo chmod -R 777` para dar permisos totales a la carpeta de datos y evitar bloqueos al mover archivos. Al revisar los logs del contenedor, descubro que el script `organizador.py` no arranca porque he mezclado espacios y tabulaciones en el código, provocando un error de indentación.

[illegible]

Reinicio del entorno para aplicar cambios

Lanzo un docker-compose down para detener y borrar los contenedores que daban fallos y luego uso up -d para volver a levantarlos de cero . Hago esto para que el sistema se refresque y cargue por fin el script de la IA con las correcciones que acabo de hacer .

```
adahi@trabajoia:~/nas_ia$ cd ~/nas_ia
docker-compose down
docker-compose up -d
Stopping nas_ia_brain          ... done
Stopping e888a27633cc_nas_samba ... done
Removing nas_ia_brain         ... done
Removing e888a27633cc_nas_samba ... done
Removing network nas_ia_default
Creating network "nas_ia_default" with the default driver
Creating nas_ia_brain ... done
Creating nas_samba     ... done
adahi@trabajoia:~/nas_ia$
```

Configuración de contraseñas Samba y reconstrucción

Ejecuto el comando smbpasswd dentro del contenedor para asignar manualmente la contraseña de red a mi usuario y asegurar el acceso desde Windows . Después, lanzo de nuevo el comando de construcción para que la IA se actualice con los últimos cambios del script que acabamos de corregir .

```
adahi@trabajoia:~/nas_ia$ docker exec -it nas_samba smbpasswd -a adahi
New SMB password:
Retype new SMB password:
adahi@trabajoia:~/nas_ia$ cd ~/nas_ia
docker-compose up -d --build
Building ia
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  8.192kB
Step 1/6 : FROM python:3.9-slim
--> 2d97f6910b16
Step 2/6 : RUN apt-get update && apt-get install -y      tesseract-ocr      tesseract-ocr-spa      libtesseract-dev      && rm -rf /var/lib/apt/lists/*
--> Using cache
--> 76fcc8a209af
Step 3/6 : RUN pip install pytesseract watchdog Pillow
--> Using cache
--> 66fd629777fb
Step 4/6 : WORKDIR /app
--> Using cache
--> e868c6969361
Step 5/6 : COPY organizador.py .
--> 5f73274796c0
Step 6/6 : CMD ["python", "organizador.py"]
--> Running in 6839b0ec4193
```

Confirmación del estado healthy de Samba

Vuelvo a lanzar un docker ps para verificar que el servidor Samba está finalmente estable y en estado "healthy". Con esto me aseguro de que el contenedor ya no se reinicia y que los puertos están escuchando correctamente para recibir archivos desde Windows .

```
adahi@trabajoia:~/nas_ia$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS                               NAMES
af100319818e  dperson/samba "/sbin/tini -- /usr/"   5 minutes ago  Up 5 minutes (healthy)  0.0.0.0:139->139/tcp, [::]:139->139/tcp, 137-138/udp, 0.0.0.0:445->445/tcp, [::]:445->445/tcp  nas_samba
```

Actualización de herramientas de orquestación

Realizo un `sudo apt update` e instalo la versión 2 de Docker Compose (`docker-compose-v2`) para asegurarme de contar con la versión más estable y moderna de la herramienta. Con esto busco mejorar la compatibilidad entre los contenedores y evitar posibles errores de sintaxis en el archivo YAML al levantar los servicios de la IA y el NAS.

```
adahi@trabajoia: ~/nas_ia
adahi@trabajoia:~/nas_ia$ sudo apt update
sudo apt install docker-compose-v2 -y
Obj:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Obj:2 http://archive.ubuntu.com/ubuntu noble InRelease
Obj:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Leyendo lista de paquetes... 5%
```

Resolución de conflictos y limpieza de contenedores

Al intentar levantar los servicios con la nueva versión de Docker Compose, el sistema me devuelve un error de conflicto: el nombre `nas_ia_brain` ya está siendo usado por un contenedor anterior. Para solucionarlo, ejecuto `docker rm -f` para forzar el borrado del contenedor conflictivo y luego utilizo `docker compose down` para limpiar por completo la red y los servicios del proyecto, dejando el entorno listo para un despliegue limpio.

```
adahi@trabajoia:~/nas_ia$ docker compose up -d --build
[0000] /home/adahi/nas_ia/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 71.1s (11/11) FINISHED
=> [ia internal] load build definition from Dockerfile
=> [ia internal] load metadata for docker.io/library/python:3.9-slim
=> [ia internal] load .dockerignore
=> [ia internal] load build context
=> [ia internal] load metadata for docker.io/library/python:3.9-slim
=> [ia 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabdca1e13cf1731b1b
=> [ia 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabdca1e13cf1731b1b
=> [ia 2/5] RUN apt-get update && apt-get install -y tesseract-ocr tesseract-ocr-spa libtesseract-dev && rm -rf /var/lib/apt/lists/*
=> [ia 3/5] RUN pip install pytesseract watchdog Pillow
=> [ia 4/5] WORKDIR /app
=> [ia 5/5] COPY organization.py .
=> [ia] exporting to image
=> [ia] exporting layers
=> [ia] exporting manifest sha256:c8f07212a634a695a897c8a62c75d5a237699a43625dc8bdc6a974a080e5
=> [ia] exporting config sha256:e804face75642bfa67ed20723c16e011a2401386468e8fc7a7f34461451e408
=> [ia] exporting attestation manifest sha256:b2e937375d2cc30cd49c7f202c37a4c945061e7d0e0cb5434c60a370ba6d8c
=> [ia] exporting manifest list sha256:d21f6b8b59b7c68811fd10476aa5cafd26ac20a1fc3d17760100f73991103fbb
=> [ia] naming to docker.io/library/nas_ia-ia:latest
=> [ia] unpacking to docker.io/library/nas_ia-ia:latest
=> [ia] resolving provenance for metadata file
[+] Running 2/3
✔ ia Built 0.0s
✔ Container nas_samba Recreate 0.0s
✖ Container lc684d7ff5c5_nas_ia_brain Error response from daemon: Conflict. The container name "/lc684d7ff5c5_nas_ia_brain" is already in use by container "lc684d...
Error response from daemon: Conflict. The container name "/lc684d7ff5c5_nas_ia_brain" is already in use by container "lc684d7ff5c53740f83d752869206381422f84713e341e4b5099bb656b480838". You have to remove (or rename) that container to be able to reuse that name.
adahi@trabajoia:~/nas_ia$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
af100319810e   dperson/samba "/sbin/tini -- /usr/..." 10 minutes ago Up 10 minutes (healthy) 0.0.0.0:139->139/tcp, [::]:139->139/tcp, 137-138/udp, 0.0.0.0:445->445/tcp, [::]:445->445/tcp nas_samba
adahi@trabajoia:~/nas_ia$ # Forzar el borrado del contenedor que da problemas
docker rm -f nas_ia_brain

# Detener y limpiar todo el proyecto actual
docker compose down
Error response from daemon: No such container: nas_ia_brain
[0000] /home/adahi/nas_ia/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 3/3
✔ Container lc684d7ff5c5_nas_ia_brain Removed 0.0s
✔ Container nas_samba Removed 0.3s
✔ Network nas_ia_default Removed 0.1s
adahi@trabajoia:~/nas_ia$
```


Despliegue definitivo y exitoso de los servicios

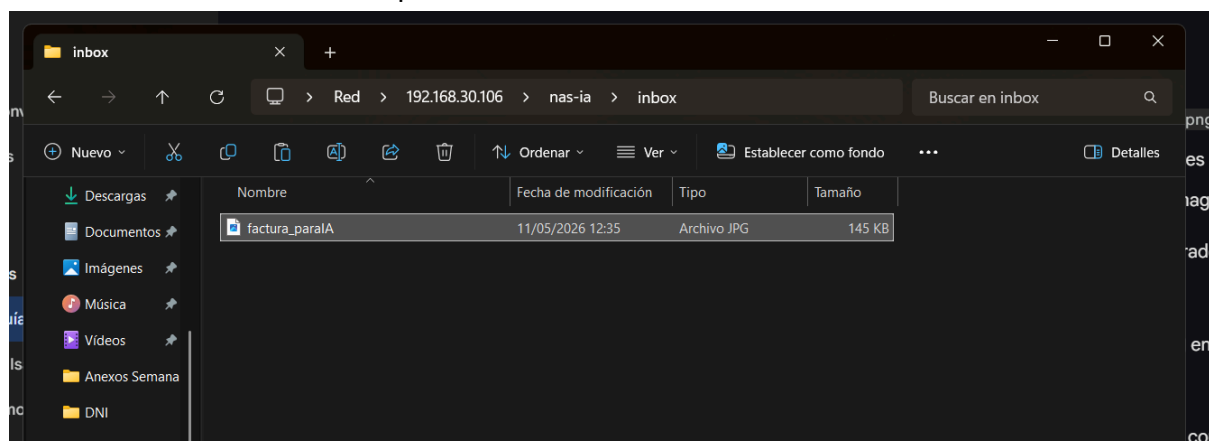
Ejecuto nuevamente docker compose up -d --build. Tras haber limpiado los conflictos anteriores, el sistema construye la imagen de la IA aprovechando las capas en caché y levanta correctamente tanto la red como los dos contenedores: nas_samba y nas_ia_brain. Los checkmarks verdes confirman que todo el entorno está finalmente en marcha y operativo, listo para procesar archivos.

```
adahi@trabajoia: ~/nas_ia
adahi@trabajoia:~/nas_ia$ docker compose up -d --build
[0000] /home/adahi/nas_ia/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 0.6s (11/11) FINISHED
=> [ia internal] load build definition from Dockerfile
=> [ia internal] load metadata for docker.io/library/python:3.9-slim
=> [ia internal] load .dockerignore
=> [ia internal] transferring context: 20
=> [ia 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3860f261f53f144965f755599aabb1acda1e13cf1731b1b
=> [ia internal] load build context
=> [ia internal] transferring context: 36B
=> CACHED [ia 2/5] RUN apt-get update && apt-get install -y tesseract-ocr tesseract-ocr-spa libtesseract-dev && rm -rf /var/lib/apt/lists/*
=> CACHED [ia 3/5] RUN pip install pytesseract watchdog Pillow
=> CACHED [ia 4/5] WORKDIR /app
=> CACHED [ia 5/5] COPY organizador.py .
=> [ia] exporting to image
=> [ia] exporting layers
=> [ia] exporting manifest sha256:ac8fb7212a634e695a897c8a62e75d5aa237699a43625dccc0ad4e0e74e809e5
=> [ia] exporting config sha256:edd4face756420bfa67eda20723c16e011a2401186468e8fc7af50401453e408
=> [ia] exporting attestation manifest sha256:6b72045206a697c0b99232dc0c0686e41241e6066404180678977ff47e2c8
=> [ia] exporting manifest list sha256:897f98943ed07fc58d912c91f5bec1428310e954091ffa43aaf5bc4917492d6a
=> [ia] naming to docker.io/library/nas_ia:latest
=> [ia] unpacking to docker.io/library/nas_ia:latest
=> [ia] resolving provenance for metadata file
[+] Running 4/4
✔ ia Built
✔ Network nas_ia_default Created
✔ Container nas_samba Started
✔ Container nas_ia_brain Started
adahi@trabajoia:~/nas_ia$
```

Fase 5: Pruebas de Clasificación Automática y Validación Real

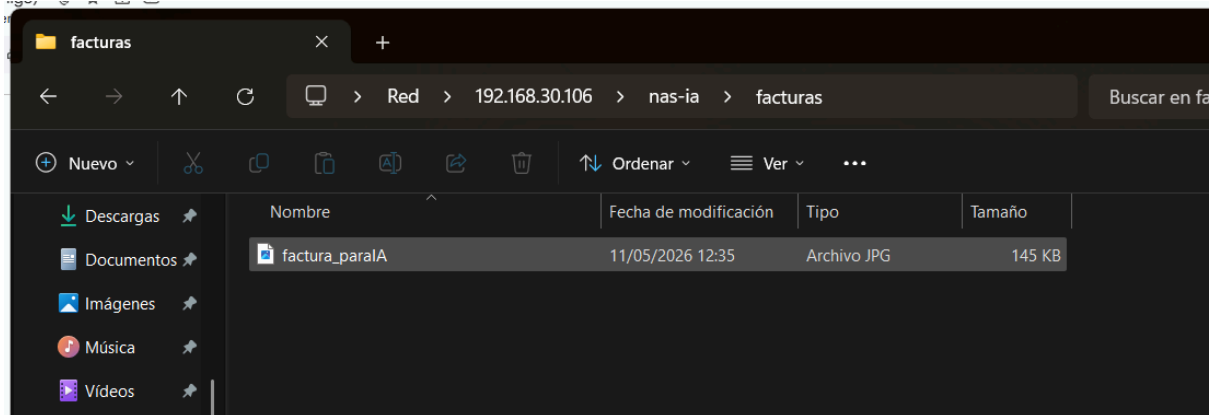
Prueba de carga en la carpeta 'inbox'

Llegó la prueba real. Subo manualmente un archivo JPG llamado factura_paraIA a la carpeta de entrada (inbox) a través de la red de Windows. En este punto, el sistema ya está vigilando y listo para que el "cerebro" de la IA entre en acción en cuanto detecte que el archivo se ha terminado de copiar.



Éxito en la clasificación automática

Compruebo que el archivo factura_paraIA ya no está en la carpeta de entrada, sino que la IA lo ha movido automáticamente a la carpeta facturas. Esto demuestra que el script ha sido capaz de leer el contenido de la imagen, identificar el tipo de documento y organizarlo correctamente en el NAS sin intervención manual. Con esto, el sistema de organización inteligente queda totalmente validado.



Optimización de logs en tiempo real

Realizo un último ajuste en el Dockerfile añadiendo el flag -u al comando final. Esto fuerza a Python a usar una salida "unbuffered", lo que significa que puedo ver los logs de la IA en tiempo real sin que se queden atascados en el búfer de Docker. Es ese pequeño detalle técnico que marca la diferencia cuando quieres vigilar qué archivos está moviendo el "cerebro" al instante.

```
adahi@trabajoia: ~/nas_ia
GNU nano 7.2 /home/adahi/nas_ia/Dockerfile *
FROM python:3.9-slim

# Instalar el motor de IA Tesseract y librerías de sistema
RUN apt-get update && apt-get install -y \
    tesseract-ocr \
    tesseract-ocr-spa \
    libtesseract-dev \
    && rm -rf /var/lib/apt/lists/*

# Instalar librerías de Python
RUN pip install pytesseract watchdog Pillow

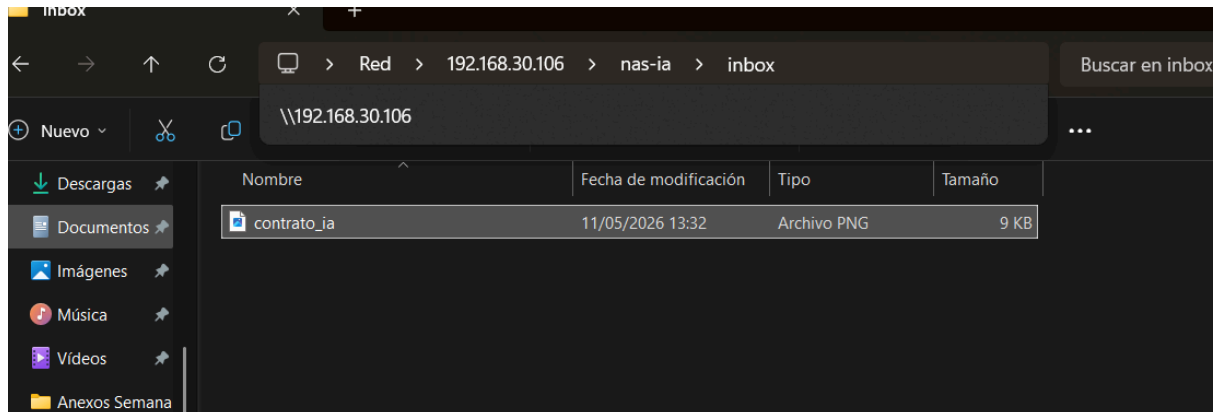
# Crear directorio de trabajo
WORKDIR /app

# Copiar el script
COPY organizador.py .

# Ejecutar el script
CMD ["python", "-u", "organizador.py"]
```

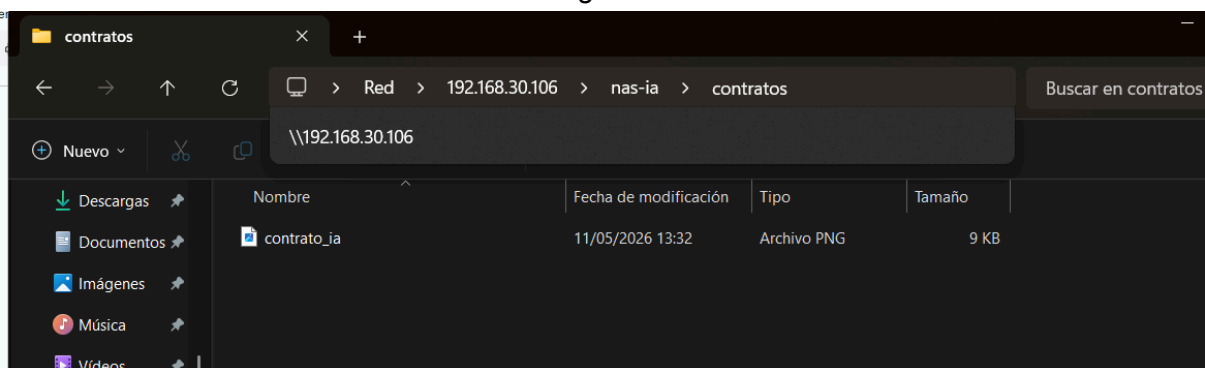
Segunda prueba de clasificación con contratos

Para terminar de validar la versatilidad del sistema, realizo una segunda prueba subiendo un archivo llamado contrato_ia a la carpeta inbox. El objetivo es confirmar que la lógica de la IA es capaz de discriminar entre distintos tipos de documentos; en este caso, deberá reconocer las palabras clave específicas de un contrato para enviarlo a su carpeta correspondiente, separándolo de las facturas procesadas anteriormente.



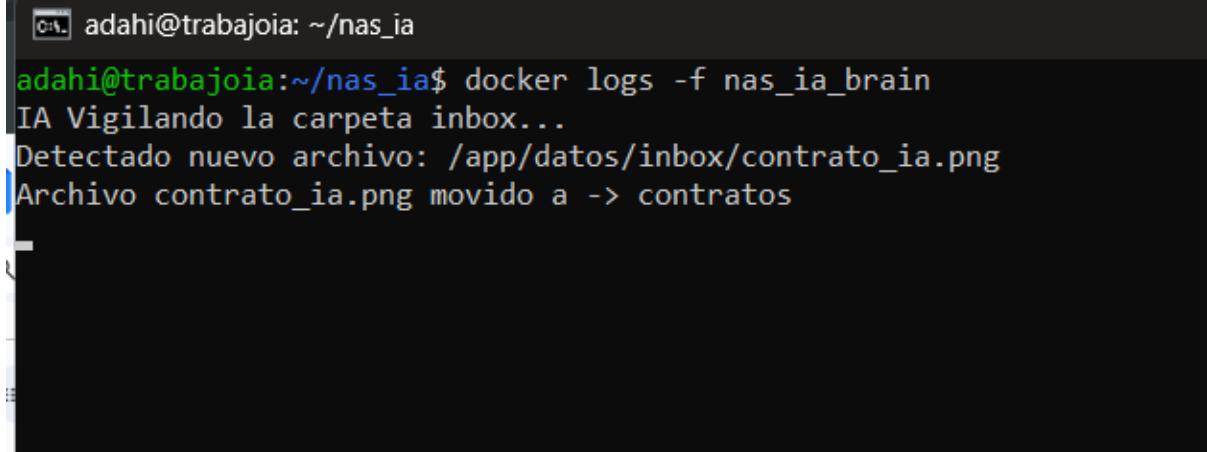
Verificación de la clasificación de contratos

Entré en la carpeta de contratos y compruebo que el archivo contrato_ia ha sido movido correctamente. Con esto queda demostrado que la IA es capaz de diferenciar entre categorías y organizar cada documento en su sitio correspondiente de forma totalmente autónoma. El sistema es ahora un NAS inteligente 100% funcional.



Monitorización de logs y confirmación de la IA

Muestro los logs del contenedor nas_ia_brain en tiempo real mediante el comando docker logs -f. En la terminal se puede ver claramente cómo la IA confirma que está vigilando la carpeta, detecta la llegada del archivo contrato_ia.png y ejecuta la acción de moverlo a la carpeta de destinos tras procesarlo. Es la prueba definitiva de que todo el flujo de trabajo automático está funcionando como un reloj.



```
adahi@trabajoia: ~/nas_ia
adahi@trabajoia:~/nas_ia$ docker logs -f nas_ia_brain
IA Vigilando la carpeta inbox...
Detectado nuevo archivo: /app/datos/inbox/contrato_ia.png
Archivo contrato_ia.png movido a -> contratos
```